

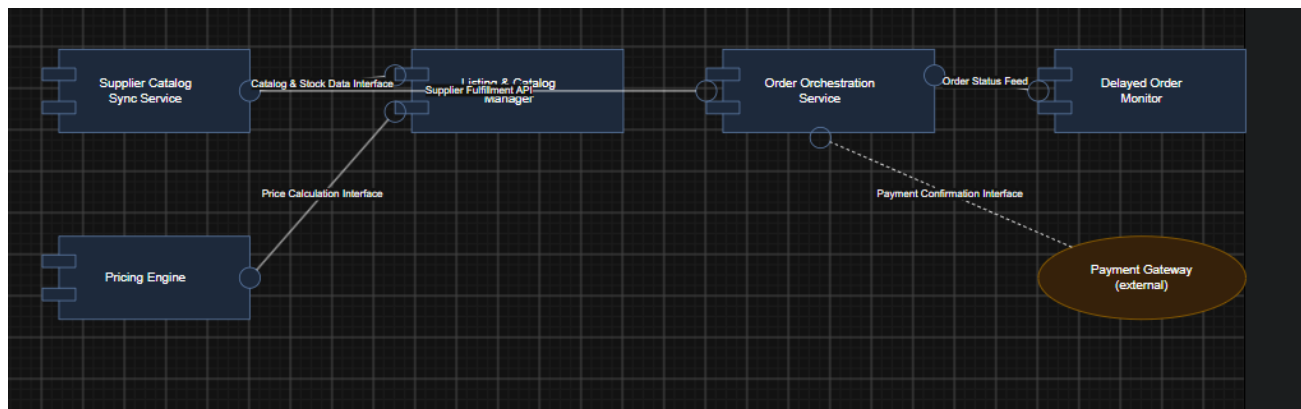
SE LAB -3

SRN: PES1UG24AM249

NAME: SANJANA M KULKARNI

1. COMPONENT DIAGRAM

For the Dropshipping Inventory & Order Orchestrator, I broke the system down into five components, since that's roughly where the natural seams in the requirements fell. The Supplier Catalog Sync Service handles the constant polling from every supplier feed, the Listing & Catalog Manager is what the store owner actually interacts with when publishing products, the Pricing Engine works out the retail price and margin, the Order Orchestration Service pushes confirmed orders out to the right supplier, and the Delayed Order Monitor keeps an eye on anything running late. The diagram below shows how they connect, plus the external Payment Gateway that triggers the order-forwarding step.



Architecture Selection: We chose Microservices Architecture for the Dropshipping Inventory & Order Orchestrator.

Reason 1: The system has to stay connected to a whole roster of Supplier Partners, each pushing stock and pricing updates on its own schedule. If that syncing logic lived in the same layer as order processing, a slow or misbehaving supplier feed could end up dragging down order forwarding too. Splitting the Supplier Catalog Sync Service out on its own means it can scale, retry, or fail independently without touching the rest of the system.

Reason 2: Looking at FR-001 and FR-004, the inventory sync workload and the order-forwarding workload behave nothing alike — one is a constant background trickle of updates, the other is a burst of near-real-time requests right after payment. Keeping them as separate, independently deployable services lets each be scaled and tuned for its own traffic pattern instead of forcing both into one shared stack.

Security Advantage: Because the Order Orchestration Service is the one handling payment-confirmed order data, isolating it as its own microservice lets us apply the stricter encryption and authentication controls required by NFR-001 right at that service's boundary — without having to lock down lower-risk services like catalog browsing to the same degree.

Performance Benefit: Since Inventory Sync runs as a separate service from Order Orchestration, heavy supplier-feed polling never blocks the order-forwarding path. That separation is what actually makes it

realistic to hit the sub-500ms order-forwarding target in NFR-001, even while the sync service is busy pulling updates in the background.